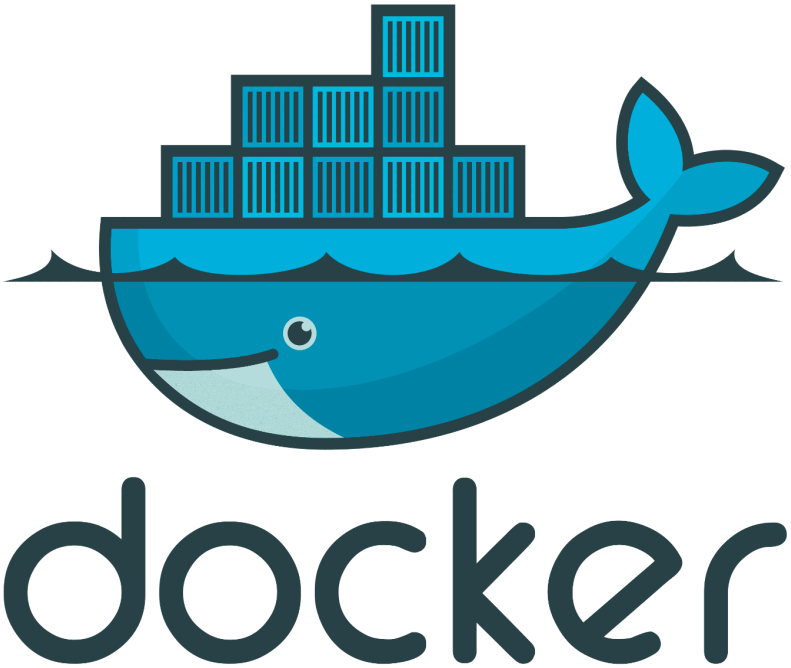
DICCIONARIO DE DOCKER



DOCKER: LA ENCICLOPEDIA TOTAL

De la ingeniería del Kernel a la orquestación de élite.

Consta de: Un desglose técnico absoluto en 8 tomos que cubre desde la creación de imágenes optimizadas y seguridad avanzada (Hardening), hasta el despliegue de microservicios, IA y clústeres a escala global con Kubernetes.

Dirigido a: Desarrolladores, Arquitectos y DevOps que no buscan un tutorial rápido, sino dominar la infraestructura de contenedores al nivel de los expertos que diseñan los sistemas de las grandes tecnológicas.

CAPÍTULO 1: FUNDAMENTOS Y ARQUITECTURA

1.1 Origen, Filosofía y el Problema del Software Moderno (Parte I)

1.1.1 La Crisis de la Matriz de Dependencias

Para comprender la necesidad de Docker, es imperativo analizar el estado del desarrollo de software previo a 2013. El problema fundamental no residía en la escritura del código, sino en su portabilidad y ejecución consistente. A este fenómeno se le denominó en la industria como "The Matrix from Hell" (La Matriz del Infierno).

Imaginemos una arquitectura estándar que requiere los siguientes componentes:

1. Un servidor web (Nginx o Apache).
2. Un intérprete de lenguaje (Python, Ruby, Node.js).
3. Una base de datos (PostgreSQL, MySQL).
4. Servicios de caché (Redis).
5. Colas de tareas (RabbitMQ).

Cada uno de estos componentes posee dependencias del sistema operativo (bibliotecas compartidas .so en Linux o .dll en Windows). El conflicto surge cuando el entorno de desarrollo (la máquina local del programador) difiere del entorno de producción.

Ejemplo Técnico del Conflicto: El desarrollador utiliza Python 3.8.1 en un entorno macOS. El servidor de producción ejecuta Debian 10 con Python 3.6 preinstalado por el sistema. La aplicación utiliza una función de la librería asyncio que fue modificada entre la versión 3.6 y la 3.8.

- Resultado: La aplicación falla en producción con un RuntimeError que es irreproducible en local.

Antes de la contenerización, la solución implicaba el uso de herramientas de gestión de configuración como Ansible, Puppet o Chef. Sin embargo, estas herramientas operan sobre el sistema operativo *mutándolo*. Si dos aplicaciones en el mismo servidor requieren versiones diferentes de la misma librería (ej: libssl), se produce un conflicto de dependencias irresoluble sin aislamiento.

1.1.2 Evolución de la Infraestructura: De Bare Metal a Contenedores

La industria ha pasado por tres fases críticas de abstracción de hardware para solucionar el aislamiento:

Fase 1: Bare Metal (Ejecución Nativa) Se instala el sistema operativo (SO) directamente sobre el hardware.

- Ventaja: Máximo rendimiento. Acceso directo a CPU/RAM.
- Desventaja: Cero aislamiento. Si una aplicación tiene una fuga de memoria, tumba el servidor completo. Tiempos de despliegue lentos.

Fase 2: Virtualización (Máquinas Virtuales - VMs) La solución predominante hasta la llegada de Docker. Se utiliza un software denominado Hypervisor (como VMware ESXi, KVM o VirtualBox). El Hypervisor permite dividir un servidor físico en múltiples servidores virtuales.

- Arquitectura de una VM:
    1. Hardware del Host.
    2. Sistema Operativo del Host (opcional en Hypervisors Tipo 1).
    3. Hypervisor.
    4. Sistema Operativo Invitado (Guest OS): Cada VM tiene su propio Kernel, drivers y binarios.
    5. Aplicación.
- El problema de la VM: El desperdicio de recursos (Overhead). Para ejecutar una aplicación de 10 MB, es necesario levantar un Sistema Operativo completo que puede pesar 2 GB y consumir 512 MB de RAM solo para mantenerse encendido. El tiempo de arranque se mide en minutos.

Fase 3: Contenerización (Docker) Docker propone un cambio de paradigma: Virtualización a nivel de Sistema Operativo, no de Hardware. En lugar de instalar un SO completo para cada aplicación, los contenedores comparten el Kernel del Host.

- Arquitectura de un Contenedor:
    1. Hardware del Host.
    2. Sistema Operativo del Host.
    3. Container Engine (Docker Daemon): Intermediario que asigna recursos.
    4. Binarios/Librerías (solo lo necesario).
    5. Aplicación.

La diferencia crítica es que el contenedor es un proceso aislado dentro del sistema operativo anfitrión. No simula hardware; solicita recursos directamente al Kernel. Esto reduce el *overhead* a casi cero y permite tiempos de arranque en milisegundos.

1.1.3 El Concepto de Inmutabilidad

Docker fuerza una práctica conocida como Infraestructura Inmutable. En el modelo tradicional, si se necesitaba actualizar una librería, un administrador de sistemas entraba por SSH al servidor y ejecutaba apt-get upgrade. Esto generaba "Configuration Drift" (desviación de configuración): con el tiempo, nadie sabía el estado exacto del servidor.

En Docker, los contenedores son efímeros e inmutables.

- No se actualizan: Si necesitas cambiar una librería, no entras al contenedor a cambiarla. Modificas la imagen base, construyes una nueva versión del contenedor, destruyes el anterior y arrancas el nuevo.
- Garantía de Estado: Esto asegura que lo que se probó en el entorno de Test es, bit a bit, idéntico a lo que corre en Producción.

1.1.4 La Estandarización: El Fin de la "Guerra de Contenedores" y la OCI

Cuando Docker lanzó su versión 1.0 en 2014, revolucionó la industria, pero también generó un monopolio de facto. Otras empresas, como CoreOS (con su runtime rkt), intentaron competir creando sus propios formatos de contenedores. Esto amenazaba con fragmentar el ecosistema: una imagen creada para Docker no funcionaría en Kubernetes o en rkt, y viceversa.

Para evitar este caos, en 2015 se fundó la Open Container Initiative (OCI) bajo el amparo de la Linux Foundation. El objetivo era simple pero ambicioso: crear un estándar abierto de la industria para los contenedores.

La OCI define dos especificaciones críticas que todo ingeniero debe conocer:

1. image-spec (Especificación de Imagen): Define cómo se debe empaquetar una imagen en el disco. Determina el formato de los archivos (layers), el manifiesto (JSON que describe la imagen), y la configuración.
    - Consecuencia: Gracias a esto, puedes construir una imagen con una herramienta que no sea Docker (como Buildah o Kaniko) y Docker podrá ejecutarla sin problemas, siempre que cumpla la image-spec.
2. runtime-spec (Especificación de Tiempo de Ejecución): Define cómo se debe desempaquetar una imagen y ejecutarla. Establece cómo se configuran los Namespaces, Cgroups y el sistema de archivos raíz.
    - Consecuencia: Permite que existan diferentes "motores" (runtimes) de bajo nivel. El estándar de facto aquí es runc.

1.1.5 Desglosando el Monolito: La Arquitectura Modular de Docker

En sus inicios, Docker era un monolito. Un solo binario hacía todo: descargaba imágenes, las construía, ejecutaba el servidor API y lanzaba los procesos. Esto era peligroso: si el daemon de Docker se actualizaba o fallaba, todos los contenedores en ejecución morían.

Para solucionar esto y adherirse a la filosofía UNIX ("hacer una cosa y hacerla bien"), Docker se refactorizó en componentes independientes. Hoy, cuando instalas Docker, estás instalando en realidad cuatro piezas de software distintas que trabajan en cadena:

A. El Docker Daemon (dockerd)

Es el proceso de alto nivel y larga duración.

- Función: Escucha las peticiones de la API de Docker (CLI o REST API). Gestiona objetos complejos como volúmenes, redes e imágenes.
- Rol: Actúa como el "cerebro" orquestador, pero no toca directamente los contenedores. Delega esa tarea a containerd.

B. containerd (El Supervisor)

Originalmente parte de Docker, ahora es un proyecto graduado de la CNCF (Cloud Native Computing Foundation).

- Función: Gestiona el ciclo de vida completo del contenedor: descarga la imagen (pull), gestiona el almacenamiento y la red, y le dice al runtime de bajo nivel que arranque el contenedor.
- Importancia: Kubernetes usa containerd directamente, saltándose a Docker para ser más ligero.

C. runc (El Ejecutor)

Es la implementación de referencia de la OCI runtime-spec. Es una herramienta de línea de comandos ligera y efímera.

- Función: Su única tarea es hablar con el Kernel del Sistema Operativo para crear los Namespaces y Cgroups necesarios y lanzar el proceso.
- Comportamiento: Una vez que el contenedor arranca, runc se cierra. No se queda en ejecución. Esto es vital para la estabilidad.

D. containerd-shim (El Intermediario)

Si runc se cierra al arrancar el contenedor, ¿quién vigila al proceso? Aquí entra el shim.

- Función: Es un proceso diminuto que se queda "pegado" a cada contenedor en ejecución.
    1. Permite que el contenedor siga vivo aunque reinicies el Docker Daemon (actualizaciones sin downtime).
    2. Recoge los códigos de salida (exit codes) del contenedor para reportarlos al daemon.
    3. Mantiene abiertos los flujos de entrada/salida (STDIN/STDOUT) para los logs.

APARTADO 1.2: ARQUITECTURA PROFUNDA DEL KERNEL (LA "MAGIA" DE LINUX)

1.2.1 La Gran Mentira: El Contenedor "No Existe"

Desde el punto de vista del Kernel de Linux, no existe ninguna estructura de datos llamada "contenedor". Si buscas en el código fuente del Kernel, no encontrarás una struct container.

Lo que llamamos "contenedor" es en realidad una ficción compartida, construida mediante la combinación de dos primitivas del Kernel que llevan existiendo años antes que Docker: Namespaces (Espacios de Nombres) y Cgroups (Grupos de Control).

- Namespaces: Se encargan del Aislamiento (Lo que el proceso puede ver).
- Cgroups: Se encargan de la Restricción (Lo que el proceso puede usar).

1.2.2 Namespaces: Los Muros de la Prisión

Cuando ejecutas un contenedor, Docker solicita al Kernel que cree un nuevo proceso con un conjunto de "etiquetas" especiales (flags) en la llamada al sistema clone(). Estas etiquetas activan los Namespaces, creando una realidad alternativa para ese proceso.

Existen 7 Namespaces fundamentales que componen las paredes de esta prisión:

A. PID Namespace (Process ID)

- Función: Aísla la lista de procesos.
- La Ilusión: Dentro del contenedor, tu aplicación cree que es el PID 1 (el primer proceso del sistema, el "padre" de todos).
- La Realidad: En el sistema Host, ese proceso puede tener el PID 45320.
- Importancia: El PID 1 tiene responsabilidades especiales (manejo de señales SIGTERM, SIGINT). Si el PID 1 muere, el Kernel mata todo el espacio de nombres (el contenedor se detiene).

B. NET Namespace (Networking)

- Función: Aísla la pila de red (tarjetas de red, reglas de firewall iptables, tablas de enrutamiento).
- La Ilusión: El contenedor cree que tiene su propia tarjeta eth0 y su propia IP (ej: 172.17.0.2).
- La Realidad: Docker crea un par de interfaces virtuales (veth pair). Un extremo queda dentro del contenedor (eth0) y el otro se conecta al "Bridge" del host (docker0). Es como un cable de red virtual que atraviesa el Kernel.

C. MNT Namespace (Mount)

- Función: Aísla los puntos de montaje del sistema de archivos.
- La Ilusión: El proceso ve un sistema de archivos raíz (/) completamente diferente al del host. Puede ver /etc/debian_version aunque el host sea Fedora.
- Mecanismo: Es una evolución del antiguo comando chroot, pero mucho más seguro y potente. Permite montar volúmenes sin que el proceso sepa que vienen de otro disco.

D. UTS Namespace (UNIX Time-Sharing)

- Función: Aísla el *hostname* y el *domainname*.
- La Ilusión: Permite que el contenedor tenga su propio nombre de máquina (ej: web-server-01) independiente del nombre real del servidor. Esto es vital para que las aplicaciones no choquen entre sí en la red.

E. IPC Namespace (Inter-Process Communication)

- Función: Aísla los recursos de comunicación entre procesos, como la memoria compartida (System V IPC) y las colas de mensajes POSIX.
- Consecuencia: Un proceso en un contenedor no puede escribir en la memoria compartida de un proceso en otro contenedor (a menos que se configure explícitamente). Evita ataques de lectura de memoria cruzada.

F. USER Namespace

- Función: Aísla los IDs de usuario (UID) y grupo (GID).
- La Ilusión (y Seguridad): Permite que un proceso sea root (UID 0) dentro del contenedor, pero que en el Host sea un usuario sin privilegios (UID 1000).
- Nota Crítica: Por defecto, Docker no activa esto en todas las instalaciones por complejidad. Si no usas User Namespaces, el root del contenedor es el root del host (peligroso si escapa).

1.2.3 Control Groups (Cgroups): El Presupuesto de Recursos

Si los Namespaces son los muros que impiden ver a los vecinos, los Cgroups son el contador de la luz y el agua. Sin ellos, un solo contenedor podría consumir el 100% de la CPU y la RAM, tumbando al servidor entero (el famoso problema del "vecino ruidoso").

Los Cgroups organizan los procesos en jerarquías y aplican límites:

A. Limitación de Memoria (Memory Controller)

- Configuración: `docker run --memory="512m"`
- Funcionamiento: El Kernel vigila cada página de RAM que pide el proceso.
- El OOM Killer (Out Of Memory): Si el contenedor intenta usar 513MB, el Kernel interviene y mata el proceso instantáneamente para proteger al resto del sistema. Verás un error `OOMKilled` (Exit Code 137).

B. Limitación de CPU (CPU Controller)

- Configuración: `docker run --cpus="1.5"`
- Funcionamiento: No significa que usa 1.5 núcleos físicos. Significa que, en cada ciclo de reloj del procesador (scheduler), el contenedor recibe tiempo de ejecución equivalente a 1.5 CPUs.
- Shares vs Quotas: Docker usa CFS (Completely Fair Scheduler) para asignar microsegundos de tiempo de CPU.

C. Limitación de E/S de Disco (Blkio Controller)

- Configuración: Permite limitar la velocidad de lectura/escritura (IOPS o Bps) en el disco.
- Uso: Vital para evitar que un contenedor de base de datos sature el disco duro y deje lentos a los demás contenedores.

D. PIDs Controller

- Defensa: Limita cuántos sub-procesos puede crear un contenedor. Evita ataques de tipo "Fork Bomb" (un virus que se replica infinitamente hasta colgar el sistema).

1.2.4 Union File System (UnionFS): El Suelo Mágico

El tercer pilar no es un sistema de aislamiento, sino de almacenamiento. Un contenedor necesita un sistema de archivos. Copiar 1GB de Ubuntu cada vez que lanzas un contenedor sería lento e ineficiente.

Solución: Copy-on-Write (CoW) Docker utiliza drivers de almacenamiento (siendo overlay2 el estándar actual en Linux) que permiten superponer capas.

1. LowerDir (Imagen): Son capas de solo lectura. (Ubuntu Base + Librerías). Son inmutables.
2. UpperDir (Contenedor): Es una capa fina y vacía de lectura/escritura que se pone encima.
3. Merged (Vista Unificada): El proceso ve la unión de ambas.

La Magia del CoW: Si el contenedor quiere modificar un archivo que está en la imagen base (ej: /etc/hosts), Docker no modifica el original. Copia ese archivo a la capa superior (UpperDir) y lo modifica ahí. El archivo original queda oculto ("masked") por la nueva versión.

- Resultado: El arranque es instantáneo porque no se copian datos, solo punteros.

APARTADO 1.3: EL ECOSISTEMA DOCKER Y LA CAJA DE HERRAMIENTAS

Para dominar Docker, no basta con saber que existen contenedores; hay que entender los componentes que permiten que esos contenedores viajen por el mundo. Este apartado detalla el flujo de trabajo profesional.

1.3.1 Los Tres Grandes Componentes: Client, Host y Registry

La arquitectura de Docker se basa en un modelo Cliente-Servidor. Esto es fundamental: cuando escribes un comando, no se ejecuta en tu terminal, sino que tu terminal le envía una orden a un servidor (Daemon).

1. El Cliente (Docker CLI): Es el binario docker que ejecutas en la terminal. Su única función es aceptar tus comandos, traducirlos a formato JSON y enviarlos mediante una API REST al Docker Daemon.
    - Dato Pro: Puedes tener el cliente en tu laptop y controlar un Docker Daemon que está en un servidor en la otra punta del mundo (usando la variable DOCKER_HOST).
2. El Host (Docker Daemon - dockerd): Es el servidor donde viven los contenedores. Gestiona las imágenes, las redes y los volúmenes. Como vimos en el capítulo anterior, es el que coordina a containerd y runc.
3. El Registry (El almacén): Es donde se guardan las imágenes. El más famoso es Docker Hub, pero las empresas usan registros privados por seguridad.
    - Flujo: Tú haces un build en tu Host, haces un push al Registry, y luego en Producción haces un pull del Registry al servidor final.

1.3.2 Docker Desktop vs. Docker Engine: La Gran Diferencia

Muchos desarrolladores se confunden aquí. Dependiendo de tu sistema operativo, Docker funciona de forma radicalmente distinta:

A. Docker Engine (Linux Nativo)

En Linux, Docker corre "a pelo". No hay intermediarios. Los contenedores usan el Kernel de tu propio ordenador directamente. Es la forma más eficiente y la que se usa en el 100% de los servidores de producción.

B. Docker Desktop (Windows y macOS)

Aquí hay un truco. Como Windows y Mac no tienen un Kernel de Linux, Docker no puede correr de forma nativa.

- ¿Cómo funciona?: Docker Desktop levanta una Micro-Máquina Virtual extremadamente ligera (usando Hyper-V en Windows o Apple Hypervisor en Mac) que corre un Linux minimalista.
- WSL2 (Windows Subsystem for Linux): En Windows moderno, Docker usa WSL2, lo que hace que esa máquina virtual sea casi imperceptible y muy rápida, pero sigue estando ahí.
- Consecuencia técnica: En Mac/Windows, "localhost" dentro de Docker es gestionado por una capa de red especial para que parezca que estás en Linux, pero a veces da problemas de rendimiento con el sistema de archivos.

1.3.3 Objetos Principales de Docker (El Diccionario de Entidades)

En el "universo" de Docker, todo lo que creas es un Objeto. Estos son los que el Daemon debe gestionar:

1. Imágenes: Plantillas de solo lectura. Son archivos binarios empaquetados.
2. Contenedores: Instancias ejecutables de una imagen.
3. Redes (Networks): Conductos virtuales que permiten que los contenedores se comuniquen entre sí o con el exterior.
4. Volúmenes (Volumes): Unidades de almacenamiento que sobreviven a la destrucción de los contenedores.
5. Plugins: Extensiones para añadir funcionalidades (ej: drivers de almacenamiento en la nube).

1.3.4 Los Registries: Más allá de Docker Hub

Un error común es pensar que solo existe Docker Hub. En un entorno profesional, aprenderás a trabajar con otros:

- Public Registries: Docker Hub, GitHub Container Registry (ghcr.io), Quay.io.
- Cloud Registries (Privados):
    - AWS ECR: (Elastic Container Registry).
    - Google GCR/AR: (Artifact Registry).
    - Azure ACR: (Container Registry).
- Self-Hosted: Puedes instalar tu propio registro privado usando la imagen oficial registry o herramientas como Sonatype Nexus o JFrog Artifactory.

1.3.5 El Ciclo de Vida del Desarrollo con Docker (The Workflow)

Para que tu libro sea práctico, este es el flujo que seguirás el 100% de las veces:

1. Code: Escribes tu aplicación en tu editor favorito.
2. Dockerfile: Escribes la "receta" de cómo empaquetar esa app.
3. Build: Ejecutas docker build. El Daemon crea una Imagen.
4. Ship: Ejecutas docker push. La imagen se guarda en el Registry.
5. Run: En cualquier servidor, ejecutas docker run (o usas Kubernetes). El servidor descarga la imagen y lanza el Contenedor.

APARTADO 1.4: GESTIÓN DE IMÁGENES (EL DICCIONARIO DE COMANDOS)

Si el contenedor es la "instancia", la imagen es la verdadera unidad de software. Dominar Docker empieza por saber gestionar estas imágenes con precisión quirúrgica.

1.4.1 Anatomía de un Nombre de Imagen (The Fully Qualified Name)

Cuando escribes docker pull nginx, Docker en realidad está completando el nombre por ti. Una imagen completa tiene esta estructura:

registry_url / username / repository_name : tag

- Registry URL: Por defecto es docker.io (Docker Hub).
- Username: El dueño de la imagen. Si es una imagen oficial (como nginx o python), no lleva nombre de usuario.
- Repository name: El nombre del software.
- Tag (Etiqueta): El número de versión o variante. Si no pones nada, Docker asume :latest.

> REGLA DE ORO: Nunca uses :latest en producción. Es una "etiqueta móvil". Lo que hoy es latest, mañana puede ser una versión nueva que rompa tu código. Usa siempre versiones específicas (ej: node:18.16.0-alpine).

1.4.2 El Ciclo de Vida: Comandos Esenciales

Aquí tienes el desglose técnico de los comandos para gestionar el almacenamiento de imágenes:

A. Obtención y Exploración

- docker pull <imagen>: Descarga las capas de la imagen desde el registry al host.
- docker images (o docker image ls): Muestra todas las imágenes descargadas.
    - Atributos clave: IMAGE ID (el hash único), CREATED (cuándo se creó la imagen, no cuándo la descargaste) y SIZE.
- docker inspect <imagen>: Devuelve un JSON gigante con toda la metadata (puertos expuestos, variables de entorno, arquitectura del procesador, etc.).

B. Limpieza y Mantenimiento

Las imágenes ocupan mucho espacio. Un buen ingeniero mantiene su sistema limpio:

- docker rmi <imagen>: Borra una imagen del disco local. No puedes borrar una imagen si hay un contenedor (aunque esté apagado) que la esté usando.
- docker image prune: El comando de "pánico". Borra todas las imágenes que no tienen nombre (imágenes dangling o huérfanas) que quedan tras fallos de construcción.

- docker image prune -a: Borra todas las imágenes que no estén siendo usadas por al menos un contenedor vivo.

1.4.3 El Sistema de Capas (Layering) en Profundidad

Este es el concepto más importante para entender la eficiencia de Docker. Cada comando en un Dockerfile (FROM, RUN, COPY) crea una capa.

¿Cómo funcionan las capas?

1. Solo Lectura: Todas las capas de una imagen son inmutables (no se pueden cambiar).
2. Identificabilidad: Cada capa tiene un ID propio (SHA256).
3. Reutilización: Si tienes 5 imágenes que empiezan con FROM debian:11, Docker solo guarda la capa de Debian una vez en tu disco. Esto ahorra gigas de almacenamiento.

Comando Pro: docker history <imagen> Este comando te permite ver "el pasado" de una imagen. Te muestra cada capa, qué comando la creó y cuánto pesa. Es vital para detectar por qué una imagen es demasiado grande.

1.4.4 Creación y Etiquetado (Build & Tag)

Para que tu aplicación se convierta en imagen, usamos: docker build -t mi-app:v1.0 .

- -t: Significa "tag". Le pones nombre y versión.
- .: Es el Contexto de Construcción. Le dice a Docker que use los archivos de la carpeta actual.

El comando docker tag: No crea una imagen nueva, crea un alias. Es como un acceso directo. docker tag mi-app:v1.0 mi-usuario/mi-app:v1.0 Esto es necesario antes de hacer un push, ya que el nombre debe incluir tu nombre de usuario del registry.

1.4.5 Formatos y Arquitecturas (Multi-arch)

Hoy en día, no todos los ordenadores son iguales (Intel/AMD vs Apple M1/M2 con ARM).

- Una imagen construida para x86_64 no funcionará en un servidor ARM de AWS.
- Solución: Docker soporta imágenes multi-arquitectura. Una sola etiqueta (ej: python:3.9) puede contener dentro diferentes binarios para diferentes procesadores. Docker descarga automáticamente el que corresponde a tu CPU.

APARTADO 1.5: CONTENEDORES EN EJECUCIÓN (EL MANUAL OPERATIVO)

Un contenedor es la ejecución de una imagen con una capa de lectura/escritura encima. En este apartado desglosamos cómo pasar del "archivo" a la "acción".

1.5.1 El Comando Maestro: docker run

Este es el comando más complejo de Docker porque combina tres pasos en uno: pull (si no tienes la imagen), create (prepara el contenedor) y start (lo arranca).

Sintaxis Pro: docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Las "Flags" (Banderas) que debes memorizar:

1. -d (Detached): Ejecuta el contenedor en segundo plano. Si no lo pones, tu terminal se quedará "secuestrada" por los logs del contenedor.
2. -it (Interactive + TTY): Te permite interactuar con el contenedor. Esencial si quieres entrar a la terminal del contenedor (ej: /bin/bash).
3. --name: Ponle un nombre humano. Si no lo haces, Docker le asignará uno aleatorio ridículo (como boring_wozniak).
4. -p (Publish): Mapeo de puertos Host:Container.
    - Ejemplo: -p 8080:80. El mundo ve el puerto 8080, pero dentro del contenedor la app escucha en el 80.
5. --rm: Contenedor efímero. Cuando se detenga, Docker lo borrará automáticamente. Ideal para pruebas rápidas.
6. -e (Env): Define variables de entorno.
    - Ejemplo: -e DB_PASSWORD=1234.

1.5.2 Estados de un Contenedor (Ciclo de Vida)

Un contenedor no está solo "encendido" o "apagado". Pasa por varios estados:

- Created: El contenedor está configurado pero el proceso interno no ha empezado.
- Running: El proceso principal (PID 1) está activo.
- Paused: El proceso está congelado en la RAM. No consume CPU, pero sigue ocupando memoria.
- Exited: El proceso terminó (con éxito o con error). Los archivos del contenedor siguen en disco.
- Dead: Un estado de fallo crítico, normalmente por problemas de recursos o archivos corruptos.

1.5.3 Inspección y Debugging (¿Qué está pasando ahí dentro?)

Cuando algo falla, un experto usa estas tres herramientas:

A. Los Logs

docker logs -f <nombre_o_id>

- La flag -f (follow) hace que los logs se actualicen en tiempo real. Es el "ojo" del desarrollador.

B. El comando exec (La cirugía)

A veces necesitas entrar al contenedor para ver una configuración o probar una conexión. docker exec -it <nombre_o_id> sh (o bash si está disponible).

- Importante: exec no crea un contenedor nuevo; crea un nuevo proceso dentro de un contenedor que ya está corriendo.

C. El comando inspect

docker inspect <nombre_o_id>

- Te da la dirección IP interna del contenedor, dónde están montados los volúmenes y exactamente qué comando se ejecutó al arrancar.

1.5.4 Gestión de la Salida y Reinicio

¿Qué pasa si el contenedor falla o el servidor se reinicia? Usamos Restart Policies:

- --restart no: (Por defecto) No reinicia el contenedor.
- --restart always: Siempre lo reinicia, sin importar por qué se detuvo.
- --restart on-failure: Solo lo reinicia si el código de salida indica un error (distinto de 0).
- --restart unless-stopped: Lo reinicia siempre, a menos que tú lo hayas detenido manualmente.

1.5.5 ¿Cómo detener y borrar correctamente?

1. **docker stop**: Envía una señal **SIGTERM** al proceso. Le da 10 segundos para cerrarse ordenadamente (guardar datos, cerrar conexiones).
2. **docker kill**: Envía una señal **SIGKILL**. Lo cierra instantáneamente. Puede corromper datos.
3. **docker rm**: Borra el contenedor del disco. Solo funciona si el contenedor está detenido (a menos que uses **-f**).

Tip de limpieza: **docker ps -a** muestra todos los contenedores, incluidos los apagados. Si ves muchos en estado "Exited", están ocupando espacio. Usa **docker container prune** para borrarlos todos de golpe.

APARTADO 1.6: PERSISTENCIA DE DATOS (EL MOTOR DE ALMACENAMIENTO)

En Docker, los contenedores son efímeros. Esto significa que cualquier archivo creado dentro de un contenedor se escribe en una capa temporal (Writable Layer). Si el contenedor se borra (docker rm), esa capa desaparece y los datos se pierden para siempre.

Para evitar esto, Docker ofrece tres mecanismos de persistencia, cada uno con una arquitectura distinta en el sistema de archivos del host.

1.6.1 La Capa de Escritura (The Copy-on-Write Layer)

Antes de entender los volúmenes, hay que entender por qué son necesarios.

- Cuando un proceso en un contenedor escribe un archivo, el driver de almacenamiento (usualmente overlay2) busca el archivo en las capas de la imagen (solo lectura).
- Si lo encuentra, lo copia a la capa superior (lectura/escritura).
- Problema de Rendimiento: Esta gestión de capas añade latencia. No es apta para bases de datos con alto tráfico de escritura (como MySQL o MongoDB).
- Problema de Ciclo de Vida: Los datos están ligados al ID del contenedor.

1.6.2 Named Volumes (Volúmenes con Nombre)

Es el mecanismo preferido y gestionado totalmente por Docker.

- Ubicación: En Linux, suelen vivir en /var/lib/docker/volumes/.
- Filosofía: "Docker se encarga". Tú no necesitas saber dónde están los archivos físicamente; solo le das un nombre al volumen.

Ventajas Técnicas:

1. Independencia del Host: Puedes migrar volúmenes entre diferentes servidores o nubes fácilmente.
2. Seguridad: Los procesos del host (fuera de Docker) no deberían tocar estas carpetas.
3. Drivers: Puedes usar drivers para que el volumen no esté en tu disco, sino en la nube (S3, Azure Files, NFS).

Comandos de Gestión:

- docker volume create mi_data: Crea el almacén.
- docker run -v mi_data:/var/lib/mysql mysql: Conecta el volumen mi_data a la carpeta interna de la DB.
- docker volume inspect mi_data: Te muestra la ruta real en el disco del host.
- docker volume prune: Borra todos los volúmenes que no estén siendo usados por ningún contenedor (cuidado aquí).

1.6.3 Bind Mounts (Montajes de Enlace)

Es el método de la "vieja escuela" pero esencial para el desarrollo. Consiste en mapear una ruta exacta de tu ordenador a una ruta exacta del contenedor.

- Sintaxis: `-v /ruta/en/mi/pc:/ruta/en/contenedor`
- Uso Principal: Desarrollo en vivo. Mapeas tu carpeta de código fuente al contenedor. Cuando editas el código en tu VS Code, el contenedor ve el cambio al instante sin necesidad de reconstruir la imagen.

Riesgos Técnicos:

- Permisos: Si tu usuario en el host es el 1000 y el contenedor corre como root (0), puedes tener problemas de "Permission Denied".
- Dependencia del Sistema: Si tu Dockerfile depende de un Bind Mount en `C:\Users\Admin`, ese contenedor fallará en Linux.

1.6.4 Tmpfs Mounts (Memoria Volátil)

Es el menos conocido pero vital para la seguridad y velocidad.

- Concepto: Los datos se guardan en la memoria RAM del host, nunca en el disco duro.
- Uso: Guardar secretos, claves temporales o archivos de sesión que deben ser ultra rápidos y que quieres que desaparezcan si el contenedor se apaga.
- Sintaxis: `--tmpfs /app/cache`

1.6.5 Tabla Comparativa para el Diccionario

Característica	Named Volume	Bind Mount	Tmpfs
Ubicación en el Host	Gestionada por Docker	Cualquier ruta elegida por el usuario	Memoria RAM
Portabilidad	Alta	Baja (depende de la ruta del host)	Nula
Rendimiento	Alto	Muy alto (nativo)	Ultra alto (RAM)
Facilidad de Backup	Moderada (vía Docker)	Muy fácil (copia normal)	Imposible

1.6.6 El concepto de "Mount Point" y Propagación

En este nivel avanzado del libro, debes entender que cuando montas un volumen, el contenido de la carpeta original del contenedor queda oculto.

- Si el contenedor tiene archivos en `/app` y tú montas un volumen vacío en `/app`, el contenedor verá la carpeta vacía.
- Excepción: Los Named Volumes tienen un comportamiento especial. Si el volumen está vacío, Docker copiará el contenido original de la imagen al volumen la primera vez que se use. Los Bind Mounts NO hacen esto.

APARTADO 1.7: NETWORKING (COMUNICACIÓN Y AISLAMIENTO DE RED)

En el mundo de los contenedores, la red no es algo físico, sino una abstracción del Kernel de Linux. Docker utiliza Network Drivers para definir cómo se comunican los contenedores entre sí, con el host y con el mundo exterior.

1.7.1 La Arquitectura CNM (Container Network Model)

Docker implementa el CNM, que se basa en tres interfaces:

1. Sandbox: Contiene la configuración de la pila de red del contenedor (IP, rutas, DNS).
2. Endpoint: El punto de conexión (como una tarjeta de red virtual).
3. Network: La colección de endpoints que pueden comunicarse entre sí.

1.7.2 Los 4 Drivers de Red Fundamentales

Para dominar Docker, debes saber cuándo usar cada uno de estos "modos" de red:

A. Bridge Network (El Estándar)

Es la red por defecto. Docker crea un puente virtual (llamado docker0 en el host).

- Aislamiento: Los contenedores están en una subred privada (ej: 172.17.0.0/16).
- Acceso Exterior: Para que alguien entre, debes mapear puertos (-p).
- Comunicación interna: Dos contenedores en la misma red *bridge* pueden hablar entre sí por IP o, si creas una red personalizada, por su nombre de contenedor (DNS interno).

B. Host Network

El contenedor no tiene IP propia; usa directamente la IP del host.

- Rendimiento: Es el más rápido porque no hay traducción de red (NAT).
- Uso: Aplicaciones que necesitan manejar rangos de puertos masivos o donde la latencia es crítica.
- Desventaja: No hay aislamiento. Si el contenedor usa el puerto 80, nadie más en el host puede usarlo.

C. None Network

Aisla totalmente al contenedor. Solo tiene la interfaz de *loopback* (127.0.0.1).

- Uso: Tareas de computación ultra-seguras o trabajos de procesamiento de archivos por lotes que no requieren red.

D. Overlay Network (Nivel Experto)

Permite conectar contenedores que están en servidores físicos diferentes.

- Uso: Es la base de Docker Swarm y Kubernetes. Crea una red virtual que "vuela" por encima de la red física de los servidores.

1.7.3 El DNS Interno y el Service Discovery

Esta es la funcionalidad más potente de Docker. Cuando creas una red personalizada (User-defined bridge), Docker activa un servidor DNS interno.

- ¿Cómo funciona?: Si tienes un contenedor llamado db y otro llamado web, el contenedor web puede conectar a la base de datos simplemente usando host: db en su configuración, sin saber la IP.
- Importancia: Las IPs de los contenedores son volátiles (cambian si reinicias). Los nombres son permanentes.

Comandos para el Diccionario:

- docker network ls: Lista las redes disponibles.
- docker network create mi_red_privada: Crea una red segura para tu app.
- docker network connect mi_red_privada contenedor_existente: Une un contenedor a una red en caliente.
- docker network inspect mi_red_privada: Muestra qué contenedores están conectados y sus IPs.

1.7.4 Port Publishing vs. Port Exposition

Mucha gente confunde EXPOSE en el Dockerfile con -p en el comando run.

1. EXPOSE (Documentación): Solo avisa que la app escucha en ese puerto. No abre nada hacia afuera. Es como poner un cartel que dice "Tengo una puerta aquí".
2. Publish -p (Acción): Crea una regla en el Firewall del host (iptables) para redirigir el tráfico. Es como abrir la puerta y dejar pasar a la gente.

1.7.5 El Flujo de un Paquete de Red

Para tu nivel de detalle: Cuando un paquete llega al puerto 8080 del host y tú tienes `-p 8080:80`:

1. iptables en el Host intercepta el paquete.
2. Se realiza un DNAT (Destination Network Address Translation).
3. El paquete se redirige a la interfaz virtual `veth` del contenedor.
4. Llega a la interfaz `eth0` dentro del Namespace de Red del contenedor.

Tienes toda la razón. Si queremos que esto sea una Enciclopedia de Docker y un curso de alto nivel, no podemos pasar por encima de las instrucciones del `Dockerfile` como si fueran simples comandos. Cada instrucción es una directiva para el motor de almacenamiento y tiene implicaciones en la seguridad, el tamaño y la velocidad de despliegue.

Vamos a expandir el Capítulo 2 (El Dockerfile) con una profundidad quirúrgica. Este bloque está diseñado para ser estudiado, no solo leído.

CAPÍTULO 2: INGENIERÍA DE CONSTRUCCIÓN (EL DOCKERFILE ENCICLOPÉDICO)

El Dockerfile no es simplemente una lista de pasos de instalación. Es un manifiesto que instruye al Storage Driver (normalmente Overlay2) sobre cómo realizar confirmaciones (commits) en un sistema de archivos en capas.

2.1 El Motor de Construcción: BuildKit y el Grafo de Dependencias

Antes de analizar los comandos, debemos entender que el motor moderno de Docker, BuildKit, no lee el Dockerfile de forma puramente lineal. Crea un Grafo Acíclico Dirigido (DAG).

- Ejecución Paralela: Si tienes etapas de construcción (stages) que no dependen entre sí, BuildKit las ejecutará al mismo tiempo para ahorrar minutos de espera.
- Detección de Secretos: BuildKit permite el uso de --secret, lo que evita que las claves de API queden grabadas en el historial de capas de la imagen.

2.2 Desglose Exhaustivo de Instrucciones Primarias

2.2.1 FROM: La Base de Confianza y Arquitectura

Es el punto de origen. Pero en un nivel experto, elegimos basándonos en la ABI (Application Binary Interface).

- Scratch: La imagen vacía. No tiene shell, ni librerías, ni usuarios. Se usa para binarios compilados estáticamente (Go, Rust).
- Alpine vs Debian: Alpine usa musl libc, que es más estricta y ligera. Debian usa glibc, que es el estándar de la industria. Cambiar entre ellas puede romper aplicaciones que dependan de bibliotecas de C específicas.
- Digest (sha256): Para una seguridad total en producción, no uses FROM node:20. Usa el hash único de la imagen: FROM node@sha256:c7.... Esto garantiza que nadie pueda modificar la imagen base sin que tú te des cuenta.

2.2.2 RUN: Gestión de Capas y Persistencia de Errores

Cada RUN crea una capa. Si instalas algo y lo borras en otro RUN, el espacio sigue ocupado en la capa anterior.

- La técnica de la "Línea Única": Unir comandos con && no es por estética, es por peso.
- Limpieza de Caché: Siempre debes limpiar los gestores de paquetes en el mismo comando:
  - apt-get clean && rm -rf /var/lib/apt/lists/* (Debian)
  - yum clean all (RHEL/CentOS)
  - rm -rf /var/cache/apk/* (Alpine)

2.2.3 COPY y ADD: El Transporte de Datos

- COPY --chown: Indispensable. Permite cambiar el propietario de los archivos mientras se copian. Si copias como root y luego cambias el usuario con USER, tu aplicación no podrá escribir en sus propios archivos.
- ADD y las URLs: Aunque ADD puede descargar de internet, es una mala práctica. ¿Por qué? Porque si descargas un .zip con ADD, se queda en una capa para siempre. Es mejor usar RUN wget ... && unzip ... && rm ....

2.3 Instrucciones de Configuración y Metadatos

2.3.1 ARG vs ENV (El ciclo de vida de la variable)

Característica	ARG	ENV
Disponible en Build	Sí	Sí
Disponible en el Contenedor	No	Sí
Persiste en Image History	No (técnicamente)	Sí
Uso Ideal	Versiones de software para descargar	Configuración de la App (DB_HOST)

2.3.2 WORKDIR: El Sistema de Archivos Lógico

Nunca uses RUN mkdir /app && cd /app. Docker olvida el cd en la siguiente capa. WORKDIR crea la carpeta automáticamente y asegura que todas las instrucciones siguientes se ejecuten allí.

2.4 Control de Procesos: ENTRYPOINT y CMD

Este es el cerebro del contenedor.

- JSON Form vs Shell Form: Siempre usa el formato JSON ["cmd", "arg"]. Si usas el formato de texto plano, Docker envuelve tu comando en /bin/sh -c, lo que impide que las señales de Linux (como el apagado) lleguen a tu aplicación.
- Patrón Wrapper: Es común usar un script .sh como ENTRYPOINT que prepare el entorno (ej. esperar a que la base de datos esté lista) y luego ejecute la aplicación con exec "$@".

2.5 Multi-Stage Builds (La Joya de la Corona)

Esta es la técnica que reduce una imagen de 1GB a 50MB. Se basa en tener una "etapa de construcción" y una "etapa de ejecución".

Dockerfile

```
# Etapa 1: Compilación (Builder)

FROM golang:1.21-alpine AS builder

WORKDIR /app

COPY . .

RUN go build -o myapp


# Etapa 2: Producción (Runtime)

FROM alpine:latest

WORKDIR /root/

# Solo traemos el binario, nada del código fuente ni del compilador

COPY --from=builder /app/myapp .

ENTRYPOINT ["./myapp"]
```

2.6 Optimización de Construcción con Montajes (Mounts)

BuildKit introduce el flag `--mount` dentro de la instrucción `RUN`. Esto es nivel arquitecto:

1. Cache Mount: `RUN --mount=type=cache,target=/root/.cache/go-build ....` Permite que las descargas de paquetes se guarden entre diferentes construcciones de imágenes, acelerando el proceso un 90%.
2. Secret Mount: `RUN --mount=type=secret,id=mysecret ....` Permite usar una clave SSH o un token de API durante el build sin que quede rastro de él en las capas de la imagen.

2.7 Seguridad y Hardening del Dockerfile

Para que este curso sea de élite, debemos hablar de cómo evitar que hackeen tu contenedor:

1. USER Non-Root: Nunca dejes que tu app corra como root. Un escape del contenedor con root da acceso total al servidor host.
2. LABEL: Añade metadatos como el autor, la versión y la licencia. Es vital para el mantenimiento en grandes empresas.
3. HEALTHCHECK: No basta con que el proceso esté vivo. El Healthcheck verifica que la aplicación responda (ej. curl localhost:8080). Si falla, Docker puede reiniciar el contenedor automáticamente.

2.8 Resumen Técnico de Capas (Checklist de Calidad)

- [ ] ¿He usado una imagen base mínima (Alpine/Distroless)?
- [ ] ¿He agrupado los RUN para minimizar capas?
- [ ] ¿He limpiado los temporales en la misma capa que los creé?
- [ ] ¿He usado USER para no correr como root?
- [ ] ¿He implementado Multi-stage para reducir el peso final?

CAPÍTULO 3: DOCKER COMPOSE Y LA ORQUESTACIÓN LOCAL (DICCIONARIO TÉCNICO)

Docker Compose es una especificación de Infraestructura como Código (IaC). Permite definir sistemas distribuidos complejos en un solo archivo declarativo. Su función principal es abstraer la complejidad de las redes, los volúmenes y las dependencias de arranque.

3.1 La Jerarquía del Archivo YAML

El archivo docker-compose.yml se estructura en bloques raíz que el motor de Docker interpreta para crear recursos en el sistema.

3.1.1 El Bloque de Servicios (Services)

Cada servicio es un "molde" para un contenedor.

- image vs build: Un servicio puede consumir una imagen ya existente o definir su propia construcción. Si usas ambos, Compose intentará construir la imagen y le pondrá el nombre indicado en image.
- container_name: Por defecto, Compose genera nombres tipo proyecto-servicio-1. Definir un nombre fijo es útil para scripts externos, pero impide escalar el servicio con --scale.

3.1.2 El Bloque de Redes (Networks)

Es el tejido conectivo. Compose crea una red default de tipo Bridge, pero un arquitecto define redes específicas para aislar el tráfico.

- driver: Normalmente bridge para un solo host, pero puede ser overlay (Swarm) o macvlan (para que el contenedor tenga una IP real de tu router físico).

3.1.3 El Bloque de Volúmenes (Volumes)

Define el almacenamiento persistente. A diferencia de los contenedores, los volúmenes son ciudadanos de primera clase y no se borran al hacer un down.

3.2 Atributos de Servicio: El Manual de Configuración

Aquí es donde el curso se vuelve extenso. Analicemos los atributos que definen el comportamiento de cada microservicio:

3.2.1 Control de Puertos y Redes

- ports (Publicación): Conecta el mundo exterior con el contenedor.
    - Modo Corto: "8080:80" (Puerto Host : Puerto Contenedor).
    - Modo Largo: Permite definir si el puerto es tcp o udp y si debe publicarse solo en una IP específica (ej. 127.0.0.1:8080:80 para que no sea accesible desde internet, solo desde tu PC).
- networks (Alias): Dentro de una red, puedes asignar alias. Esto permite que un contenedor responda a varios nombres DNS internos.

3.2.2 Dependencias e Inteligencia de Arranque (depends_on)

Este es un punto crítico. Docker Compose arranca los contenedores en orden, pero no sabe si la aplicación dentro del contenedor está lista.

Healthcheck Condicionado:
YAML
```
services:
  web:
    build: .
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5
```

- Este nivel de detalle asegura que el servidor web no intente conectar a la base de datos hasta que Postgres confirme que acepta conexiones.

3.3 Gestión Avanzada de Datos: Volúmenes y Montajes

3.3.1 Bind Mounts vs Named Volumes en Compose

- Bind Mounts: type: bind. Mapeas una ruta de tu disco (ej. ./config:/etc/config). Es bidireccional: lo que cambias en el host cambia en el contenedor.
- Named Volumes: type: volume. Docker gestiona la carpeta en /var/lib/docker/volumes. Es mucho más rápido en sistemas de archivos de alto rendimiento.

3.3.2 Volúmenes Externos

Si tienes una base de datos con 500GB de datos, no quieres que Compose la gestione. Usas external: true. Compose solo "se conecta" a ese volumen ya existente, pero no intentará crearlo ni borrarlo.

3.4 Variables de Entorno y Configuración (Env Files)

Un error común es poner contraseñas en el archivo YAML. Un experto usa la jerarquía de variables:

1. Directas: environment: - DB_PASS=123.
2. Archivos .env: env_file: - ./secretos.env.
3. Interpolación: Puedes usar ${DB_USER} dentro del YAML. Docker Compose buscará ese valor en las variables de entorno de tu ordenador.

3.5 Extensibilidad: extends e include

Para proyectos gigantescos, el archivo YAML puede volverse inmanejable.

- extends: Permite reutilizar configuraciones de un servicio en otro (como una clase padre en programación).
- include (Novedad 2024): Permite importar otros archivos docker-compose.yml completos. Puedes tener un archivo para la infraestructura de logs, otro para la DB y otro para la App, y unirlos todos.

3.6 Operaciones de Orquestación (CLI Deep Dive)

Para que el curso sea completo, el alumno debe dominar la terminal de Compose:

- docker-compose up -d --build: Reconstruye imágenes y arranca en segundo plano.
- docker-compose down --volumes: Detiene todo y borra incluso los volúmenes (Cuidado: borra los datos).
- docker-compose logs -f --tail=100 [servicio]: Monitorización selectiva.
- docker-compose run --rm web npm test: Ejecuta un comando puntual en un contenedor nuevo basado en la configuración del servicio "web" y lo borra al terminar. Es ideal para ejecutar migraciones de base de datos o tests unitarios.

3.7 Resumen de Arquitectura: ¿Cómo se ve una red de Compose?

Cuando levantas un proyecto con Compose:

1. Se crea una interfaz de red virtual en el Host.
2. Cada servicio recibe una IP interna (ej. 172.18.0.x).
3. Se habilita un servidor DNS interno que mapea el nombre del servicio (ej. db) a esa IP dinámica.
4. Se aplican las reglas de IPTABLES para el mapeo de puertos hacia el exterior.

CAPÍTULO 4: OPERACIONES DE MISIÓN CRÍTICA, SEGURIDAD (HARDENING) Y BAJO NIVEL

Este capítulo aborda la transición de "un contenedor que corre" a una "infraestructura resiliente". Ignorar estos detalles en producción resulta en fugas de memoria, brechas de seguridad y caídas sistémicas.

4.1 Aislamiento del Kernel: Namespaces y Capabilities en Profundidad

Como vimos en el Tomo 1, Docker usa Namespaces, pero un experto debe saber manipularlos para asegurar el sistema.

4.1.1 Linux Capabilities (Privilegios Granulares)

El usuario root en Linux tiene cerca de 40 privilegios distintos (Capabilities). Docker, por defecto, solo le otorga 14 a un contenedor. Un ingeniero de seguridad debe aplicar el Principio de Mínimo Privilegio:

- CAP_CHOWN: Permite cambiar el dueño de los archivos.
- CAP_NET_BIND_SERVICE: Permite usar puertos menores al 1024.

Práctica Maestra: Quita todos los privilegios y añade solo el necesario.
Bash
```
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE nginx
```

- Esto asegura que, si un hacker toma el control de Nginx, no pueda ni siquiera cambiar la hora del sistema (CAP_SYS_TIME) ni cargar módulos en el Kernel (CAP_SYS_MODULE).

4.1.2 El Modo Privilegiado (--privileged)

Este flag desactiva todos los mecanismos de seguridad de Docker. El contenedor obtiene acceso directo al hardware del host (discos, GPUs, interfaces de red físicas). Solo se usa para correr Docker dentro de Docker (DinD) o herramientas de diagnóstico de hardware.

4.2 Hardening de la Superficie de Ataque

4.2.1 Seccomp (Secure Computing Mode)

Seccomp es un perfil de seguridad del Kernel que filtra las llamadas al sistema (syscalls).

- Un contenedor estándar tiene acceso a unas 300 syscalls. Docker aplica un perfil por defecto que bloquea unas 44 que son consideradas peligrosas (como mount(), reboot() o swapon()).
- Uso avanzado: Puedes crear un archivo JSON personalizado para bloquear incluso chmod o mkdir dentro de tu contenedor una vez que la app ha arrancado.

4.2.2 AppArmor y SELinux

Son módulos de seguridad de Linux (LSM) que Docker soporta de forma nativa.

- AppArmor: Se basa en rutas de archivos. Define qué archivos puede leer o escribir un proceso.
- SELinux: Se basa en etiquetas de seguridad (Labels). Es el estándar en entornos Red Hat/CentOS y es mucho más estricto pero complejo de configurar.

4.3 Gestión de Recursos (Cgroups v2) y QoS

Un contenedor sin límites es una bomba de tiempo. Si un proceso entra en un bucle infinito de memoria, el Kernel de Linux activará el OOM Killer (Out of Memory Killer).

4.3.1 Gestión de Memoria (RAM y Swap)

- --memory (Hard limit): Si el contenedor llega a este límite, es eliminado.
- --memory-reservation (Soft limit): Es una garantía. El contenedor siempre tendrá esa memoria disponible, pero puede usar más si el host tiene RAM libre. Cuando el host se queda sin RAM, Docker obliga a los contenedores a bajar a su límite de reserva.
- --memory-swap: Controla cuánta memoria puede volcar el contenedor al disco duro. En bases de datos, esto debe estar desactivado para evitar latencias extremas.

4.3.2 Ciclos de CPU y CFS Scheduler

Docker no "asigna núcleos", asigna tiempo de ejecución.

- --cpus="1.5": Significa que de cada segundo de tiempo de CPU, el contenedor puede usar 1.5 segundos (repartidos en varios núcleos).
- --cpu-shares: Define la prioridad. Si dos contenedores compiten por el mismo núcleo, el que tenga más "shares" ganará más tiempo de procesamiento.

4.4 Monitorización Forense y Telemetría

Para gestionar 100 contenedores, no puedes usar docker ps. Necesitas telemetría profunda.

4.4.1 El Socket de Docker (/var/run/docker.sock)

Es la puerta trasera del motor.

- Peligro: Si montas este archivo dentro de un contenedor, ese contenedor puede crear, borrar y ver todos los demás contenedores del host. Es como darle las llaves maestras de la casa.
- Uso legítimo: Herramientas de monitorización como Portainer o Prometheus lo necesitan para leer estadísticas.

4.4.2 Análisis de Logs con Drivers

En producción, no usamos el driver json-file (el que lee docker logs) porque llena el disco.

- Driver syslog: Envía los logs a un servidor central.
- Driver fluentd o gelf: Envía los logs directamente a un stack ELK (Elasticsearch, Logstash, Kibana) para análisis de errores en tiempo real.

4.5 Mantenimiento de Higiene del Sistema de Archivos

Docker utiliza el Storage Driver Overlay2. Este driver crea capas en /var/lib/docker/overlay2. Con el tiempo, se acumulan "capas huérfanas".

4.5.1 El comando system prune a fondo

- docker system prune -a --volumes: El comando definitivo. Borra:
    1. Contenedores detenidos.
    2. Redes no usadas.
    3. Imágenes sin contenedores asociados (incluso las que tienen nombre).
    4. Volúmenes (CUIDADO: esto borra tus bases de datos si no están respaldadas).

4.5.2 Inspección de Espacio (docker system df)

Te muestra exactamente cuánto espacio ocupan:

- Imágenes.
- Contenedores (capa de escritura).
- Volúmenes locales.
- Caché de construcción (BuildKit).

4.6 Hardening de Red: El puente docker0 e Iptables

Cuando Docker arranca, modifica las reglas de Iptables de tu servidor Linux.

- NAT (Network Address Translation): Docker mapea las IPs privadas de los contenedores a la IP pública del host.
- IP Forwarding: Debe estar activado en el Kernel (sysctl net.ipv4.ip_forward=1) para que los contenedores tengan internet.
- Seguridad: Si quieres bloquear el acceso de un contenedor a una IP específica de tu empresa, debes hacerlo en las reglas de Iptables del host, no dentro del contenedor.

CAPÍTULO 5: EL CICLO DE VIDA DE PRODUCCIÓN (CI/CD, REGISTRIES Y AUTOMATIZACIÓN)

En la ingeniería de software moderna, el contenedor no es el fin, sino el vehículo. Este capítulo analiza la infraestructura necesaria para que el código pase de la mente del desarrollador al servidor de producción de forma determinista, segura y escalable.

5.1 Integración Continua (CI) a Bajo Nivel

La CI es el proceso de transformar código fuente en un artefacto inmutable. En Docker, esto implica el uso de Builders especializados.

5.1.1 El Motor BuildKit y el Protocolo LLB

BuildKit no es solo un comando; es un motor que traduce el Dockerfile a Low-Level Builder (LLB), un formato binario de grafo de estados.

- Ejecución Acíclica: BuildKit analiza qué pasos pueden ejecutarse en paralelo. Si tienes dos etapas de compilación independientes, las ejecutará simultáneamente aprovechando todos los núcleos del servidor de CI.
- Detección de Archivos No Cambiados: A diferencia del motor antiguo, BuildKit solo envía al daemon los archivos necesarios para la capa actual, reduciendo drásticamente el uso de red.

5.1.2 Pruebas de Humo (Smoke Tests) y Testeo en Contenedor

Un curso de élite exige validar la imagen antes de subirla:

- Test Kitchen / Goss: Herramientas que arrancan el contenedor recién construido y verifican: ¿El puerto 80 responde? ¿El binario /app/server tiene permisos de ejecución? ¿Existe el usuario non-root?
- Sidecar Containers en CI: Durante la fase de test, el servidor de CI levanta la imagen de la App y, en paralelo, un contenedor de Base de Datos para asegurar que la integración es real.

5.2 Estrategias de Versionado y el "Manifiesto de Verdad"

El etiquetado (tagging) es la metainformación más crítica en la orquestación.

5.2.1 La Jerarquía de Etiquetas Inmutables

Para evitar el desastre de que una actualización automática rompa el sistema, se utiliza un sistema de etiquetas redundantes:

1. Tags de Desarrollo: feat-login-shadow.
2. Tags de Versión (SemVer): v2.4.1.
3. Tags de Entorno: staging, production (estas son etiquetas móviles que apuntan a un hash específico).
4. Immutable Digests: El uso del SHA256 (image@sha256:...). Es la única forma de garantizar que el código que corre en el servidor es exactamente el mismo que se testeó en la CI, ya que las etiquetas de texto pueden ser sobrescritas, pero los hashes no.

5.3 Arquitectura Multi-Plataforma y Emulación (QEMU)

Con la llegada de procesadores ARM a los centros de datos (AWS Graviton, Google Tau), la compilación cruzada es obligatoria.

5.3.1 Docker Buildx y los Nodos de Construcción

docker buildx extiende las capacidades de Docker para manejar múltiples instancias de construcción.

- Emulación via binfmt_misc: Permite que un kernel Linux x86 ejecute instrucciones ARM mediante QEMU. Aunque es más lento, permite generar imágenes para Raspberry Pi o servidores Cloud ARM desde una laptop estándar.
- Remote Nodes: Un arquitecto configura un "Build Farm". Cuando lanzas un build, Buildx distribuye el trabajo: la parte Intel la compila un servidor Xeon y la parte ARM la compila un servidor Ampere, uniendo los resultados en un solo manifiesto.

5.4 Distribución Global: Registries y Replicación

El Registry no es solo un "almacén", es un nodo de red crítico.

5.4.1 Anatomía de un Registro de Grado Empresarial (Harbor)

Para una enciclopedia, debemos mencionar a Harbor (proyecto de la CNCF):

- Replicación basada en políticas: Permite que una imagen subida en Madrid se replique automáticamente en Singapur y Nueva York en milisegundos.
- Proxy Cache: Si tus servidores están en una red privada, el Registry actúa como caché de Docker Hub para evitar descargar 100 veces la misma capa de Nginx desde internet.
- Vulnerability Gate: Puedes configurar el registro para que impida el Pull de cualquier imagen que tenga vulnerabilidades críticas detectadas por el escáner (Trivy o Clair).

5.5 Optimización de la Cadena de Suministro (Supply Chain Security)

5.5.1 SBOM (Software Bill of Materials)

Es un archivo JSON (en formatos como SPDX o CycloneDX) que acompaña a la imagen.

- ¿Qué contiene?: Una lista de cada librería compartida, cada paquete de Python/Node y cada binario de sistema operativo presente en la imagen.
- Utilidad: Si se descubre una vulnerabilidad en libssl.so.1.1, no tienes que escanear tus imágenes; consultas tu base de datos de SBOMs y sabes instantáneamente qué contenedores están en riesgo.

5.5.2 Firma de Imágenes con Cosign (Sigstore)

La firma digital asegura la procedencia. El servidor de producción está configurado para rechazar cualquier imagen que no tenga una firma válida del servidor de CI de la empresa. Esto evita ataques de "Man-in-the-middle" donde un atacante sustituye una imagen por una maliciosa.

5.6 Despliegue Continuo (CD) y Estrategias de Rollout

¿Cómo llega la imagen al servidor?

5.6.1 Push vs Pull (GitOps)

- Push (Tradicional): La CI ejecuta un script SSH o una API de Kubernetes para "empujar" la nueva versión. (Riesgo: el servidor de CI necesita acceso total a producción).
- Pull (GitOps - ArgoCD/Flux): El servidor de producción vigila un repositorio de Git. Cuando el archivo deployment.yaml cambia su versión de v1 a v2, el orquestador detecta la diferencia y "tira" de la imagen. Es mucho más seguro y permite auditorías completas.

3.6.2 Blue-Green y Canary Deployments

Docker permite, mediante balanceadores de carga (Nginx, Traefik), realizar despliegues quirúrgicos:

- Blue-Green: Mantienes la versión vieja (Blue) y la nueva (Green) corriendo al mismo tiempo. Si la Green falla, cambias el tráfico instantáneamente de vuelta a la Blue.
- Canary: Envías solo el 5% del tráfico a la nueva versión. Si no hay errores en los logs, vas aumentando el porcentaje hasta el 100%.

5.7 Gestión de Secretos en Tiempo de Ejecución

Nunca se deben inyectar secretos mediante ENV.

- Docker Secrets (Swarm): Los secretos se envían cifrados a los nodos y se montan en un sistema de archivos temporal en memoria (/run/secrets/) dentro del contenedor. Nunca tocan el disco duro.
- Integración con Vault: Las aplicaciones modernas arrancan, se autentican con HashiCorp Vault y descargan sus credenciales en tiempo de ejecución, eliminando la necesidad de guardar contraseñas en archivos de configuración.

Llegamos a la cumbre. El Capítulo 6 es el final del viaje del arquitecto. Aquí es donde dejamos de hablar de "un servidor" y empezamos a hablar de Clústeres: un ejército de máquinas que trabajan de forma coordinada.

En este nivel de la enciclopedia, no nos conformamos con saber qué es un clúster; vamos a diseccionar el plano de control, la malla de servicios y la resiliencia distribuida.

CAPÍTULO 6: ORQUESTACIÓN A ESCALA (SWARM, KUBERNETES Y EL ESTADO DESEADO)

La orquestación es el proceso de automatizar la gestión, el escalado y el despliegue de contenedores en un conjunto de máquinas (clúster). El paso de Docker Compose a un orquestador es el paso de una gestión manual a una Gestión Declarativa Basada en el Estado Deseado.

6.1 El Concepto de Clúster y el Plano de Control (Control Plane)

Un clúster de contenedores divide las máquinas en dos roles fundamentales:

1. Managers (Maestros): Son el cerebro. Mantienen la base de datos del estado del clúster (Raft log), gestionan la programación de tareas (scheduling) y atienden las peticiones de la API.
2. Workers (Nodos de trabajo): Son el músculo. Su única función es ejecutar contenedores (unidades de trabajo) y reportar su estado al Manager.

6.2 Docker Swarm: La Orquestación Nativa

Swarm es el orquestador integrado en Docker. Su ventaja competitiva es la simplicidad y el uso de la misma sintaxis que Docker CLI.

6.2.1 El Algoritmo de Consenso Raft

Para que un clúster sea resistente a fallos, los Managers deben estar de acuerdo.

- Quórum: Se recomienda tener un número impar de managers (3, 5 o 7). Si tienes 3 managers, el sistema aguanta que 1 muera. Si tienes 5, aguantas que mueran 2.
- Estado Deseado: Si tú defines que quieres 10 réplicas de Nginx y un servidor explota, el Manager detecta que "Estado Actual (9) != Estado Deseado (10)" y lanza automáticamente una nueva instancia en otro nodo sano.

6.2.2 Servicios, Tareas y Redes Overlay

- Service: Es la definición de alto nivel (Imagen, puertos, réplicas).
- Task: Es la ranura donde se ejecuta el contenedor. Si un servicio tiene 3 réplicas, tiene 3 tasks.
- Routing Mesh (Malla de Enrutamiento): Es una tecnología de red asombrosa. Puedes enviar una petición a la IP de cualquier nodo del clúster, y Docker la redirigirá internamente al nodo que realmente tiene el contenedor corriendo.

6.3 Kubernetes (K8s): El Estándar de la Industria

Si Swarm es un coche, Kubernetes es una flota de portaaviones. Es el orquestador desarrollado originalmente por Google para manejar miles de millones de contenedores.

6.3.1 La Anatomía de un Pod

En Kubernetes no manejamos contenedores directamente, manejamos Pods.

- Un Pod es la unidad mínima. Puede contener uno o más contenedores que comparten la misma IP (localhost) y volúmenes.
- Sidecar Pattern: Un contenedor corre la App y otro (el sidecar) corre los logs o el proxy de seguridad.

6.3.2 Objetos de Kubernetes (El Diccionario K8s)

1. Deployment: Define cómo se actualiza la app (ej. de 10% en 10%).
2. Service (ClusterIP, NodePort, LoadBalancer): Expone la app al mundo o internamente.
3. ConfigMaps y Secrets: Inyectan configuración sin reconstruir la imagen.
4. Ingress: El controlador que gestiona el tráfico HTTP/HTTPS, certificados SSL y nombres de dominio.

6.4 Alta Disponibilidad y Auto-Escalado (HPA)

En una enciclopedia de ingeniería, debemos hablar de la Elasticidad.

- Horizontal Pod Autoscaler (HPA): Kubernetes vigila el consumo de CPU. Si tu app supera el 70% de CPU debido a un pico de ventas (Black Friday), K8s crea automáticamente más Pods. Cuando el tráfico baja, los borra para ahorrar costes.
- Self-Healing (Auto-curación): Si un contenedor se queda bloqueado (Deadlock), el orquestador lo mata y lo reinicia basándose en el Liveness Probe.

6.5 Service Mesh (Malla de Servicios): El nivel Final

Cuando tienes 500 microservicios hablándose entre sí, la red se vuelve un caos. Aquí entran herramientas como Istio o Linkerd.

- mTLS Automático: Todo el tráfico entre contenedores se cifra automáticamente sin que el programador haga nada.
- Circuit Breaking: Si un microservicio empieza a fallar, la malla de servicios corta el tráfico hacia él para evitar un efecto dominó que tumbe toda la plataforma.

6.6 Conclusión de la Enciclopedia: El Futuro

Docker y los contenedores han cambiado la informática para siempre. Hemos pasado de:

1. Era del Hierro: Instalar software en servidores físicos (meses de espera).
2. Era de la Virtualización: Máquinas virtuales (minutos de espera).
3. Era del Contenedor: Microservicios efímeros (milisegundos).
4. Era Serverless: Correr código sin siquiera pensar en el contenedor (Docker corre por debajo, pero tú no lo ves).

CAPÍTULO 7: CASOS DE USO ESPECIALIZADOS (IA, GPU Y ENTORNOS HÍBRIDOS)

En este tomo final, exploramos cómo Docker se extiende más allá de los microservicios web estándar para abordar la computación de alto rendimiento y la compatibilidad con sistemas legados.

7.1 Docker para Inteligencia Artificial y GPU (NVIDIA Docker)

Tradicionalmente, los contenedores no tienen acceso al hardware físico del host (como la tarjeta gráfica). Sin embargo, para tareas de Deep Learning, Renderizado 3D o Minería de Datos, es imprescindible usar la potencia de la GPU.

7.1.1 El Runtime de NVIDIA (nvidia-container-toolkit)

Para que un contenedor pueda "ver" una GPU, no basta con instalar Docker. Se requiere el NVIDIA Container Toolkit.

- Arquitectura: El toolkit actúa como una capa que inyecta las librerías de controladores (.so en Linux) y los binarios de CUDA dentro del contenedor en tiempo de ejecución.

Uso técnico:
Bash
```
docker run --gpus all nvidia/cuda:12.0-base nvidia-smi
```

- El flag --gpus permite asignar GPUs específicas (ej. --gpus "device=0") o limitar la memoria de video disponible para ese contenedor.

7.2 Docker en Windows Server (Windows Containers)

Es vital no confundir Docker Desktop (que corre Linux en Windows) con los Windows Containers nativos. Estos últimos están diseñados para modernizar aplicaciones .NET Framework antiguas que no pueden correr en Linux.

7.2.1 Tipos de Aislamiento en Windows

1. Process Isolation (Aislamiento de Proceso): Los contenedores comparten el kernel de Windows Server. Es ultra rápido pero requiere que la versión del SO del host sea idéntica a la de la imagen.
2. Hyper-V Isolation: Cada contenedor corre dentro de una micro-máquina virtual con su propio kernel de Windows. Es mucho más seguro y permite correr imágenes de Windows 10 en un Windows Server, por ejemplo.

7.2.2 El Ecosistema IIS y .NET

Docker permite empaquetar un servidor IIS (Internet Information Services) completo. Esto permite que aplicaciones de hace 15 años se beneficien de la escalabilidad y el despliegue de Docker sin cambiar una sola línea de código C#.

7.3 Virtualización Anidada: Docker in Docker (DinD)

Existen situaciones (especialmente en servidores de CI/CD como Jenkins o GitLab Runner) donde necesitas que un contenedor de Docker sea capaz de construir y arrancar otros contenedores.

7.3.1 Los dos métodos de DinD

1. El método del Socket (/var/run/docker.sock): Se monta el socket del host dentro del contenedor. El contenedor "cree" que corre Docker, pero en realidad le está pidiendo al Docker del host que haga el trabajo. Es más rápido pero menos aislado.
2. DinD Real (--privileged): El contenedor corre su propio daemon independiente. Requiere privilegios totales y es más pesado, pero es el único método que garantiza aislamiento total entre los contenedores "hijo" y el sistema host.

7.4 Edge Computing y Arquitecturas IoT

Docker es el estándar para desplegar software en dispositivos remotos (Raspberry Pi, sensores industriales, vehículos autónomos).

- Resiliencia en Redes Inestables: Docker permite descargar imágenes por capas. Si la conexión se corta en un túnel o en alta mar, el motor de Docker retomará la descarga desde el último byte recibido cuando vuelva la señal.
- Actualizaciones Atómicas: Si una actualización falla en un dispositivo IoT a 500km de distancia, Docker permite hacer un Rollback instantáneo a la versión anterior, evitando que el dispositivo quede "ladrillo" (bricked).

7.5 Docker y el Big Data: El patrón Sidecar de Almacenamiento

En sistemas como Hadoop o Spark, mover datos es costoso. Docker permite acercar el cómputo al dato.

- Data Locality: Usando redes de alto rendimiento (como InfiniBand) y volúmenes montados directamente en buses NVMe, Docker permite procesar petabytes de información con una latencia cercana al metal (Bare Metal).

7.6 Auditoría Final: El Checklist del Arquitecto

Para cerrar esta enciclopedia, cada proyecto de Docker debe pasar por este filtro de calidad:

1. Seguridad: ¿He escaneado las imágenes? ¿Corro como usuario no-root?
2. Eficiencia: ¿He usado Multi-stage builds? ¿He optimizado la caché?
3. Resiliencia: ¿Tengo Healthchecks definidos? ¿Hay políticas de reinicio?
4. Escalabilidad: ¿Mi aplicación es Stateless (sin estado) para poder duplicarla?
5. Observabilidad: ¿Están los logs centralizados? ¿Tengo límites de recursos (CPU/RAM)?

CAPÍTULO 8: EL ABISMO TÉCNICO Y LA FRONTERA TECNOLÓGICA

Este capítulo final explora las entrañas del motor y las tecnologías emergentes que están redefiniendo el concepto de "contenedor".

8.1 Depuración a Nivel de Kernel (Tracing y Profiling)

Cuando un contenedor se comporta de forma errática pero los logs están limpios, el ingeniero senior utiliza herramientas de Observabilidad de Bajo Nivel.

8.1.1 eBPF (Extended Berkeley Packet Filter)

eBPF es la tecnología más revolucionaria en el ecosistema Linux. Permite ejecutar programas dentro del Kernel sin cambiar su código.

- Uso en Docker: Herramientas como Cilium o Falco usan eBPF para monitorizar cada llamada al sistema, cada paquete de red y cada escritura en disco de un contenedor en tiempo real, con un impacto casi nulo en el rendimiento.
- Diagnóstico: Si un contenedor muere misteriosamente, eBPF puede decirte exactamente qué syscall causó el pánico del kernel.

8.1.2 Strace y GDB dentro de Contenedores

A veces necesitas "ver" qué está haciendo el proceso paso a paso.

- Técnica avanzada: Usar nsenter para entrar en el namespace de un contenedor desde el host y ejecutar strace -p <PID>. Esto permite interceptar todas las comunicaciones entre la aplicación y el kernel sin necesidad de instalar herramientas de debug dentro de la imagen de producción.

8.2 WebAssembly (Wasm) y Docker: El Contenedor sin SO

Esta es la frontera actual. Docker ha integrado recientemente el soporte para Wasm.

- ¿Qué es?: Wasm permite ejecutar binarios en un entorno de sandbox ultra seguro y ligero.
- Diferencia con Docker tradicional: Un contenedor de Linux lleva un sistema operativo (aunque sea mínimo). Un "contenedor" Wasm solo lleva el binario.
- Ventaja: Arranca en microsegundos (100 veces más rápido que un contenedor normal) y ocupa una fracción del espacio. Docker ahora permite ejecutar ambos tipos de cargas de trabajo simultáneamente.

8.3 Almacenamiento Efímero y Optimización de IOPS

En bases de datos de alta transaccionalidad, el sistema de archivos Overlay2 es un cuello de botella debido a la "penalización de copia al escribir" (Copy-on-Write).

- Direct I/O y Raw Devices: Un experto sabe cuándo saltarse el sistema de archivos de Docker y usar montajes de tipo block device.

Tmpfs (RAM Disks): Para archivos temporales o cachés que no deben tocar el disco, Docker permite montar carpetas directamente en la memoria RAM del host:
Bash
docker run --tmpfs /app/cache:rw,size=2g my_app

- Esto reduce el desgaste de los discos SSD y acelera la aplicación de forma masiva.

8.4 Redes de Malla (Service Mesh) y Zero Trust

En el Tomo 6 hablamos de orquestación, pero el nivel "Maestro" requiere entender la Seguridad Zero Trust.

- mTLS (Mutual TLS): No basta con cifrar el tráfico. En una red Zero Trust, cada contenedor tiene su propio certificado digital. Antes de hablar, los contenedores se presentan sus "DNI digitales". Si uno está comprometido, el otro corta la comunicación.
- Sidecar Proxy (Envoy): Toda la lógica de red se saca de la aplicación y se pone en un contenedor pequeño al lado. La app cree que habla por HTTP normal, pero el proxy lo cifra y lo enruta de forma segura.

8.5 Arquitecturas "Serverless" sobre Docker

El futuro es no gestionar servidores. Tecnologías como Knative (sobre Kubernetes) permiten que Docker se comporte como una función Lambda.

- Scale-to-Zero: Si nadie usa tu aplicación, el orquestador borra todos los contenedores. En cuanto llega una petición HTTP, Docker levanta el contenedor en milisegundos, atiende la petición y vuelve a apagarse. Esto ahorra millones en facturas de Cloud.

8.6 El Manifiesto del Ingeniero de Contenedores

Para ser el mejor, debes adoptar esta filosofía:

1. **Inmutabilidad: Un contenedor nunca se parchea en caliente. Se destruye y se crea uno nuevo con la versión corregida.**
2. **Efemeralidad: Diseña tu app pensando que el contenedor puede morir en cualquier segundo. Los datos deben estar fuera (en volúmenes o bases de datos).**
3. **Transparencia: Si no puedes medirlo, no existe. Cada contenedor debe exponer métricas (/metrics).**
4. **Seguridad por Capas: No confíes en el aislamiento de Docker. Usa USER, ReadOnly FS, Capabilities y Seccomp.**

APÉNDICE FINAL: DICCIONARIO DE COMANDOS (REFERENCIA RÁPIDA)

1. Gestión de Contenedores (Ciclo de Vida)

- docker ps --> Listar solo los contenedores que están en ejecución actualmente.
- docker ps -a --> Listar todos los contenedores (activos, detenidos, con errores o terminados).
- docker run <imagen> --> Crear y arrancar un contenedor nuevo a partir de una imagen.
- docker run -d <imagen> --> Arrancar un contenedor en segundo plano (modo detached).
- docker run --name <nombre> <imagen> --> Asignar un nombre personalizado a un contenedor al crearlo.
- docker run -p 8080:80 <imagen> --> Mapear el puerto 8080 del host al puerto 80 del contenedor.
- docker start <id_o_nombre> --> Iniciar uno o más contenedores que estaban detenidos.
- docker stop <id_o_nombre> --> Detener un contenedor en ejecución de forma controlada (envía SIGTERM).
- docker kill <id_o_nombre> --> Forzar la detención de un contenedor inmediatamente (envía SIGKILL).
- docker restart <id_o_nombre> --> Detener y volver a arrancar un contenedor.
- docker rm <id_o_nombre> --> Eliminar un contenedor (debe estar detenido).
- docker rm -f <id_o_nombre> --> Forzar la eliminación de un contenedor aunque esté corriendo.
- docker pause <id_o_nombre> --> Suspender todos los procesos dentro de un contenedor.
- docker unpause <id_o_nombre> --> Reanudar los procesos de un contenedor suspendido.

2. Gestión de Imágenes

- docker images --> Listar todas las imágenes descargadas o construidas localmente.
- docker pull <imagen> --> Descargar una imagen desde un registro (como Docker Hub).
- docker push <imagen> --> Subir una imagen local a un registro remoto.
- docker build -t <nombre> . --> Construir una imagen a partir de un Dockerfile en el directorio actual.
- docker rmi <id_imagen> --> Eliminar una imagen local específica.
- docker tag <id_original> <nuevo_nombre> --> Crear una etiqueta (alias) para una imagen existente.
- docker history <imagen> --> Mostrar el historial de capas y comandos que crearon la imagen.
- docker save -o archivo.tar <imagen> --> Guardar una imagen en un archivo comprimido para moverla sin red.
- docker load -i archivo.tar --> Cargar una imagen desde un archivo comprimido .tar.

3. Diagnóstico y Depuración

- docker logs <id_o_nombre> --> Ver la salida (stdout/stderr) de un contenedor.
- docker logs -f <id_o_nombre> --> Seguir los logs del contenedor en tiempo real.
- docker exec -it <id_o_nombre> sh --> Entrar en un contenedor en ejecución con una terminal interactiva.
- docker inspect <id_o_nombre> --> Obtener toda la información técnica (JSON) de un objeto Docker.
- docker stats --> Mostrar flujo en tiempo real de uso de CPU, memoria y red de los contenedores.
- docker top <id_o_nombre> --> Listar los procesos internos que se ejecutan dentro del contenedor.
- docker port <id_o_nombre> --> Ver qué puertos del host están mapeados al contenedor.
- docker diff <id_o_nombre> --> Ver los archivos que han cambiado en el contenedor respecto a la imagen.

4. Redes y Volúmenes

- docker network ls --> Listar todas las redes configuradas en Docker.
- docker network create <nombre> --> Crear una nueva red (por defecto tipo bridge).
- docker network connect <red> <contenedor> --> Conectar un contenedor a una red específica.
- docker volume ls --> Listar todos los volúmenes de datos persistentes.
- docker volume create <nombre> --> Crear un nuevo volumen gestionado por Docker.
- docker volume inspect <nombre> --> Ver la ubicación física del volumen en el disco del host.
- docker volume rm <nombre> --> Eliminar un volumen (no debe estar en uso).

5. Mantenimiento y Limpieza (Pruning)

- docker system df --> Ver cuánto espacio de disco está usando Docker (imágenes, contenedores, caché).
- docker system prune --> Borrar contenedores parados, redes no usadas e imágenes huérfanas.
- docker system prune -a --> Borrar todo lo anterior más todas las imágenes que no tengan un contenedor.
- docker image prune --> Eliminar solo las imágenes que no tienen etiqueta (dangling images).
- docker volume prune --> Eliminar todos los volúmenes que no están siendo usados por ningún contenedor.

6. Docker Compose (Orquestación Local)

- docker-compose up --> Crear y arrancar todos los servicios definidos en el archivo YAML.
- docker-compose up -d --> Levantar los servicios en segundo plano.
- docker-compose down --> Detener y eliminar contenedores, redes e imágenes creadas por up.
- docker-compose stop --> Detener los servicios pero sin eliminarlos.
- docker-compose logs -f --> Ver los logs combinados de todos los servicios del YAML.
- docker-compose ps --> Listar el estado de los servicios gestionados por el archivo actual.
- docker-compose build --> Forzar la reconstrucción de las imágenes de los servicios.

7. Gestión del Swarm (Clúster y Orquestación)

- docker swarm init --> Convertir el nodo actual en el Manager de un nuevo clúster.
- docker swarm join --> Unir un nodo al clúster (requiere el token generado por el manager).
- docker swarm join-token worker --> Generar y mostrar el comando para unir nuevos nodos trabajadores.
- docker node ls --> Listar todas las máquinas que forman parte del clúster y su estado.
- docker service create --replicas 3 -p 80:80 nginx --> Lanzar un servicio equilibrado en 3 contenedores a través del clúster.
- docker service ls --> Ver todos los servicios activos en el clúster.
- docker service ps <nombre_servicio> --> Ver en qué nodos específicos están corriendo las tareas de un servicio.
- docker service scale <nombre>=10 --> Aumentar o disminuir el número de réplicas de un servicio al instante.
- docker service update --image <nueva_imagen> <nombre> --> Realizar una actualización progresiva (rolling update) de la imagen de un servicio.
- docker stack deploy -c file.yml <app> --> Desplegar un archivo de Docker Compose completo en un clúster de Swarm.
- docker secret create my_pw file.txt --> Almacenar un dato sensible (contraseña/llave) de forma cifrada en el clúster.

8. Seguridad y Confianza (Hardening)

- docker scout quickview --> Ver un resumen rápido de las vulnerabilidades de una imagen.
- docker scout cves <imagen> --> Listar detalladamente todas las vulnerabilidades críticas encontradas en las capas.
- docker trust inspect <imagen> --> Verificar si una imagen ha sido firmada digitalmente y quién es el autor.
- docker trust sign <imagen> --> Firmar digitalmente una imagen propia para garantizar su integridad.

9. Administración del Engine (Motor y Registros)

- docker login <servidor> --> Autenticarse en un registro privado (Docker Hub, AWS ECR, etc.).
- docker logout --> Cerrar la sesión y borrar las credenciales locales del registro.
- docker version --> Mostrar las versiones detalladas del Cliente y del Server (API, OS, Arch).
- docker info --> Ver estadísticas del sistema: cuántos contenedores hay, driver de almacenamiento y plugins.
- docker events --> Mostrar en tiempo real todas las acciones que ocurren en el motor (creación, inicio, muerte de contenedores).
- docker wait <contenedor> --> Bloquear la terminal hasta que el contenedor se detenga y devolver su código de salida.

10. Comandos Avanzados de Construcción (BuildKit)

- docker buildx ls --> Listar las instancias de constructores disponibles (para multi-arquitectura).
- docker buildx create --use --> Crear un nuevo constructor que soporte funciones avanzadas como emulación ARM.
- docker buildx build --platform linux/amd64,linux/arm64 -t user/app:v1 --push . --> Construir, empaquetar para dos CPUs distintas y subir al registro en un solo paso.
- docker builder prune --> Borrar solo la caché de construcción para liberar espacio sin borrar imágenes.

11. Plugins y Extensiones

- docker plugin ls --> Ver los plugins instalados (redes personalizadas, drivers de volumen externos).
- docker plugin install <nombre> --> Instalar una extensión nueva en el motor de Docker.

12. Atajos de Oro (Fórmulas Magistrales)

- docker stop $(docker ps -q) --> Detener todos los contenedores que están corriendo a la vez.
- docker rm $(docker ps -a -q) --> Borrar todos los contenedores del sistema de un solo golpe.
- docker rmi $(docker images -q) --> Borrar todas las imágenes locales de una vez.
- docker inspect --format '{{ .NetworkSettings.IPAddress }}' <nombre> --> Extraer directamente la dirección IP de un contenedor sin leer todo el JSON.